Follow JAVA SpringBoot Microservices WhatsApp channel :
https://whatsapp.com/channel/0029VbAU9ntBlHpTyMDC270m

==========
AI-Powered Microservices Development using Spring AI
==========

Trainer : Mr. Ashok

IT Working Exp : 12+ Years

Training Exp : 10+ Years

==========
Pre-Requisites
==========

1) Core JAVA

2) Spring Core
- IOC
- DI
- Auto Wiring

3) Spring Boot

- POM Starters
- Auto Configuration
- Embedded Server
- Acutuator
- Profiles

==========
Course Content
==========

1) Spring Web MVC

- DispatcherServlet
- HandlerMapper
- Controller
- ModelAndView
- ViewResolver

2) Spring REST

- JSON & JACKSON API
- XML & JAX-B API (outdated)
- HTTP Protocol
  - HTTP Request Format
  - HTTP Response Format
  - HTTP Methods
  - HTTP Status Codes

- REST Principles
- REST Architecture
  - Provider
  - Consumer
- Provider Development (@RestController)
  - Path Variables (@PathVariable)
  - Query Parameter (@RequestParam)
  - Request Body (@RequestBody)
  - ResponseBody (@ResponseBody)
  - ResponseEntity
  - POSTMAN
  - Swagger
  - Exception Handling
    - Controller Based

- Global Handling

- Consumer Development

- RestTemplate (outdated)
- WebClient

3) Spring Cloud with Microservices

- Monolith vs Microservices
- Microservices Architecture
- Service Registry (Eureka)
- AdminServer
- Zipkin Server
- Config Server
- APIGateway
- FeignClient (Interservice communication)
- CirbuitBreaker
- Saga Pattern (Kafka)
- Cache Pattern (Redis)

4) Spring AI

- AI vs ML vs GenAI
- Overview LLMs (Large language models)
- Real-world use cases of AI-powered microservices
- Introduction to Spring AI

- Spring AI vs OpenAI SDK vs LangChain4j
- Integrating Spring AI in a Spring Boot application

- Setting up a â€œChat Microserviceâ€ using Spring AI and OpenAI
- Understanding ChatClient and PromptTemplate
- Creating REST endpoints to interact with AI models

- What is RAG (Retrieval-Augmented Generation)

5) Spring Security

- HttpBasicAuth
- OAuth 2.0
- JWT

6) Spring Batch & Spring Schedular

==========
Course Details
==========

Timings : 8:30 AM to 10:00 AM IST

Duration : 30 to 40 days

Course Fee : 5,000 INR (live classes + videos + softcopy notes)

Note: Videos validity 1 year after course completion.

Weekly : 6 days

==========
Spring Web MVC
==========

=> One of the most famous & important module in spring framework.

=> Using Spring Web MVC module we can develop 2 types of applications.

1) Web Applications (C 2 B)

2) Distributed Application (B 2 B)

==========
Web Application
==========

=> Web Applications are used for Customer to Business Communication.

Ex: flipkart, naukri, linkedin, ashokit

Note: In Web application we will have 3 components

1) Presentation Components (UI)

2) Business Components (Controllers + Services)

3) Data Access Components (Repositories)

Note-1 : To develop presentation (UI) components in Spring Web MVC based monolithic application we can use JSP and Thymeleaf.

Note-2 : In today's world Monolithic is outdated hence we are not using JSP and Thymeleaf.

==========
What is Distributed application
==========

=> Distributed applications are called as Webservices / Rest APIs.

=> Webservices are used to communicate from one application to another application.

Ex: Passport ----------> AADHAR

Gpay --------------> SBI bank

MakeMyTrip ----------> IRCTC

=> Note: In distributed appliations UI will not be available (pure backend apis).

==========
Spring Web MVC Architecture
==========

1) Dispatcher Servlet

2) HandlerMapper

3) Controller / Request Handler

4) ModelAndView

5) ViewResolver

6) View

==========
DispatcherServlet
==========

=> It is predefined class in spring web mvc.

=> It acts as front controller (main gate of the house).

=> It is responsible to recieve request and send the response to client.

Note: It is also called as framework servlet class.

================
Handler Mapper
================

=> It is predefined class in spring web mvc.

=> It is responsible to identify controller class to handle the request based on url-pattern and give controller class details to dispatcher servlet.

===========
Controller
===========

=> Controllers are java classes which are used to handle the request (request processing).

=> DispatcherServlet will call controller class methods.

=> After processing request, controller method will return ModelAndView object to dispatcher servlet.

Model -> It is a map to represent data in key-value format

View -> It represents view page name

~~~~~~ Note: We need to develop Controller class using @Controller annotation. ~~~~~

===============
View Resolver
===============

=> It is used to identify view files location.

=> Dispatcher Servlet will give view name to View Resolver then it will identify the view file location and give it to Dispatcher Servlet.

========
View
========

=> It is responsible to render model data on the view page and give it to dispatcher servlet.

Note: DispatcherServlet will send final response to client.

==
Developing First Spring Web MVC Based App
==

Step-1 : Create boot app with below dependencies

a) web-starter

b) Thymeleaf

c) devtools

Step-2 : Create Controller class with required http methods

Step-3 : Create View Page and access Model data in view page

Views Location : src/main/resources/templates/

Step-4 : Run the application and test it using browser
~~~~~~

```
-----------------------------------------------------------

@Controller
@RequestMapping("/msg")
public class MsgController {

	@GetMapping("/welcome")
	public ModelAndView getWelcomeMsg() {

		System.out.println("getWelcomeMsg() method called...");

		ModelAndView mav = new ModelAndView();

		// data to display in view
		mav.addObject("msg", "Welcome to Spring Web MVC");

		// logical view name
		mav.setViewName("index");

		return mav;
	}

	@GetMapping("/greet")
	public String getGreetMsg(Model model) {

		System.out.println("Model impl class : " + model.getClass().getName());

		model.addAttribute("msg", "Good Morning..!!");

		return "index"; // logical view name
	}
}
-----------------------------------------------------------------------


=======================================================
How to configure jetty as default embedded container?
=======================================================

### Step-1 : Exclude tomcat from 'web-starter' in pom.xml

```
<dependency>
	<groupId>org.springframework.boot</groupId>
	<artifactId>spring-boot-starter-web</artifactId>
	<exclusions>
		<exclusion>
			<groupId>org.springframework.boot</groupId>
			<artifactId>spring-boot-starter-tomcat</artifactId>
		</exclusion>
	</exclusions>
</dependency>

```

### Step-2 : Configure jetty starter in pom.xml

```

<dependency>
	<groupId>org.springframework.boot</groupId>
	<artifactId>spring-boot-starter-jetty</artifactId>
</dependency>

```

Note-1 : Choose Tomcat if you're building standard Java EE apps with JSP, and want stability with
strong community support.
```

Note-2 : Choose Jetty if you need a lightweight, embeddable, fast-starting server for microservices or modern cloud-native apps.

==========
What is Request Parameter ?
==========

=> Request Parameters also called as Query Parameters.

=> These are used to send data from client to server in URL.

=> Request Parameters will represent data in key-value format.

Ex : https://www.youtube.com/watch?v=McmckGLzZ4Q&t=100

=> Request Parameters will start with ? and will be seperated by &.

=> To read Request Parameters from the URL in Controller class method we will use @RequestParam annotation.

```
---------------------------------------------------------------------------------
@Controller
public class MsgController {

	// URL : http://localhost:8080/greet?name=raj
	@GetMapping("/greet")
	@ResponseBody
	public String greetMsg(@RequestParam("name") String name) {

		String msg = name + ", Good Morning..!!";

		return msg;
	}

	// URL : http://localhost:8080/course?c=sbms&t=ashok
	@GetMapping("/course")
	@ResponseBody
	public String getCourse(@RequestParam("c") String course, @RequestParam("t") String
trainer) {

		String msg = course + " By " + trainer + " will start soon...";

		return msg;
	}
}
---------------------------------------------------------------------------------
```

==========
What is Path Variable ?
==========

=> Path Variables also called as URI variables and Path Parameters.

=> These are used to send data from client to server in URL.

		Ex: https://www.instagram.com/reel/{DRMfvvLD3Yd}/

Note: Path Variables will represent data directley without any key.

Note: Path Variables position we need to represent in URL template.

```
-----------------------------------------------------
// URL : http://localhost:8080/welcome/raj
@GetMapping("/welcome/{name}")
```

```
@ResponseBody
public String getWelcomeMsg(@PathVariable("name") String name) {

        String msg = name + ", Welcome to Ashok IT";

        return msg;
}
```

=> We have below limitations with URL data

1) Data is exposing in browser URL (others can read it who are sitting beside us)

2) URL length limitation

3) Will not support for binary data (ex: images, videos, audios, files etc..)

==========
What is Request Body ?
==========

=> Using Request Body we can send data from client to server without exposing in URL.

=> Using Request Body we can send binary data also (ex: images, videos, audios, files etc..) to the server.

Note: When we are submitting forms then we will use Request Body to send form data to Controller.

Note: To read data from RequestBody we will use @RequestBody annotation in controller class method.

==========
What is Form Binding ?
==========

=> In Servlets, programmer is responsible to capture form data and store that in object manually.

```
// capture form data
String name = request.getParameter("name");
String email = request.getParameter("email");
String phno = request.getParameter("phno");

// store in object
User user = new User();
user.setName(name);
user.setEmail(email);
user.setPhn(Long.parseLong(phno));
```

Note: The above is logic common for every form and in every project.

=> To avoid above problems, Spring Web MVC introduced, Form Binding concept.

=> The process of binding Java object to form fields is called as Form Binding.

=> With the help of form binding we can map java object to form fields and form fields data to java object.

Note: If we use form binding concept then DispatcherServlet will take care of capturing form data and mapping form data to java object.

=> To do form biding to java object we will use below properties in thymeleaf

th:object : ====> To represent which object mapping to form

```
                th:field : =====> To represent which field is mapped to which variable in java obj

---------------------------------------------------------------------------------

Step-1 : Create Springboot starter project with below dependencies

                - web-starter
                - thymleaf
                - devtools
                - lombok

Step-2 : Create DTO class to map form data (Form Binding class)

Step-3 : Create Controller class with required methods to handle request and response

                        Method-1 :: To load empty form

                        Method-2 :: To handle form submission


Step-4 : Create View files using bootstrap

Step-5 : Run the application and test it.

---------------------------------------------------------------------------------

================================
Email sending with Spring Boot
================================

=> To send emails we need to configure SMTP properties

                SMTP = Simple mail transfer protocol

Note: For practice purpose we can use gmail smtp properties. In Realtime project we will use
company specific SMTP properties.

Note: We need to generate gmail "app password" for SMTP authentication purpose.

                @@@ URL To Generate App Pwd : https://myaccount.google.com/apppasswords


#################### App pwd : ddjw paoo spjm stjv ####################

## Step-1 : Add "mail-starter" in pom.xml file

## Step-2 : Configure smtp properties in "application.properties" file

        spring.mail.host=smtp.gmail.com
        spring.mail.port=587
        spring.mail.username=ashokit.classes@gmail.com
        spring.mail.password=yvrn epas jjnk ksoc
        spring.mail.properties.mail.smtp.auth=true
        spring.mail.properties.mail.smtp.starttls.enable=true

## Step-3 : Use JavaMailSender to send emails.

                javaMailSender.send(message);

Note: We have 2 types of msgs to send emails

        1) SimpleMailMessage => (plain text)

        2) MimeMessage => (html body & attachments)
```